# Assignment 2 Report

**Xing Hui (Richard) Lu**
**CSCI-611 – Spring 2026**
**Assignment 2**

---

# 1. Introduction

In this assignment, I completed two tasks related to image processing and machine learning. In Part 1, I used OpenCV and convolution kernels to apply filtering techniques such as edge detection and blurring. In Part 2, I built and trained a convolutional neural network (CNN) using PyTorch to classify images from the CIFAR-10 dataset.

The goal of Part 1 was to understand how convolution kernels can modify an image by emphasizing edges or smoothing details. The goal of Part 2 was to build a CNN model from scratch, train it, and evaluate its performance on an image classification task. Overall, this assignment helped me better understand both traditional image processing techniques and deep learning-based computer vision.

---

# 2. Part 1: Image Filtering with OpenCV

## 2.1 Overview

For this part of the assignment, I applied several custom convolution kernels to a grayscale image. These filters were used for:

- horizontal edge detection
- vertical edge detection
- diagonal edge detection
- blurring with averaging filters of different sizes

I used the OpenCV function `cv2.filter2D()` to apply each kernel to the image and observe the resulting changes.

---

## 2.2 Edge Detection Kernels

The following kernels were used in this part.

### Horizontal edge detector

```
horizon = np.array([[ -1, -1, -1],
                    [  0,  0,  0],
                    [  1,  1,  1]])
```

### Vertical edge detector

```
vertica = np.array([[ -1, 0, 1],
                    [ -1, 0, 1],
                    [ -1, 0, 1]])
```

### Diagonal 45-degree detector

```
diag_45 = np.array([[ -2, -1, 0],
                    [ -1,  0, 1],
                    [  0,  1, 2]])
```

### Diagonal 135-degree detector

```
diag135 = np.array([[ 0, -1, -2],
                    [ 1,  0, -1],
                    [ 2,  1,  0]])
```

---

## 2.3 Blurring Kernels

I also used averaging filters of different sizes to blur the image.

### 2x2 averaging kernel

```
S2x2 = np.array([[1, 1],
                 [1, 1]])
```

### 3x3 averaging kernel

```
S3x3 = np.array([[1, 1, 1],
                 [1, 1, 1],
```

```
                    [1, 1, 1]])
```

### 5x5 averaging kernel

```
S5x5 = np.array([[1, 1, 1, 1, 1],
                 [1, 1, 1, 1, 1],
                 [1, 1, 1, 1, 1],
                 [1, 1, 1, 1, 1],
                 [1, 1, 1, 1, 1]])
```

Each blur kernel was normalized by dividing by the number of elements in the kernel:

- 2x2 was divided by 4
- 3x3 was divided by 9
- 5x5 was divided by 25

---

## 2.4 Results and Discussion

The edge detection filters showed how different kernels can detect features in different directions. The horizontal kernel highlighted horizontal edges, while the vertical kernel highlighted vertical edges. The diagonal kernels detected slanted edges in the image.

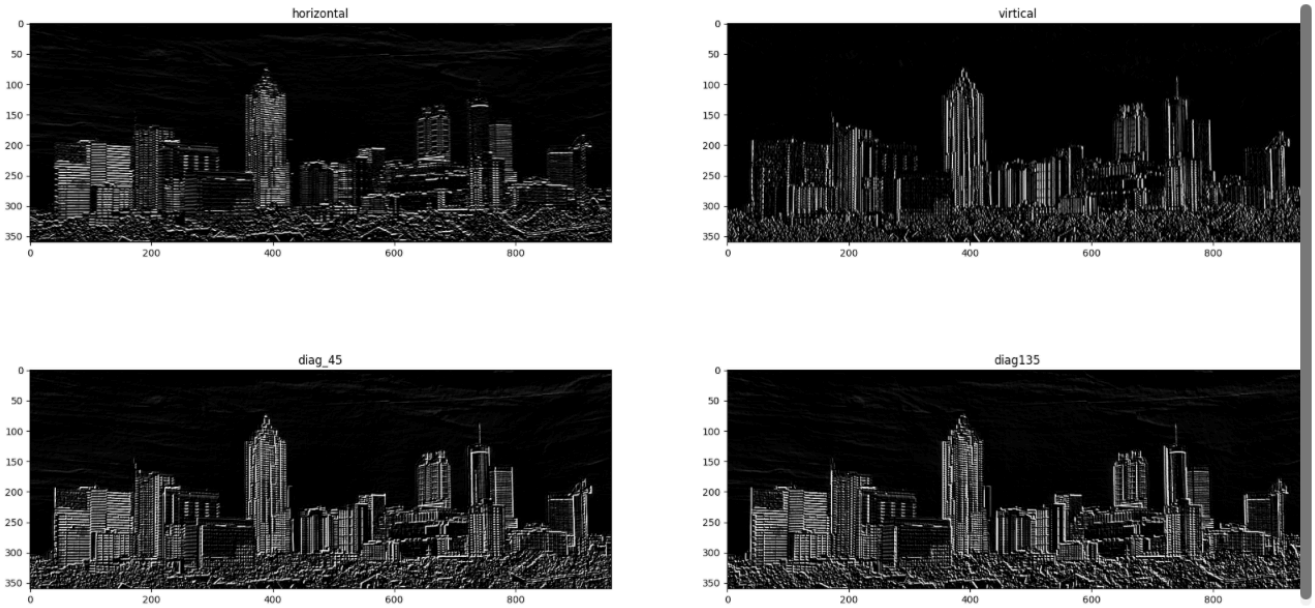
For the blurring filters, I observed that the larger the kernel size, the stronger the blur effect became. The 2x2 blur only smoothed the image slightly, the 3x3 blur made the smoothing more noticeable, and the 5x5 blur removed much more detail.

This part showed that convolution can be used both for highlighting important image features, such as edges, and for reducing detail through blurring. However, there is a tradeoff because stronger blurring also removes useful image information.

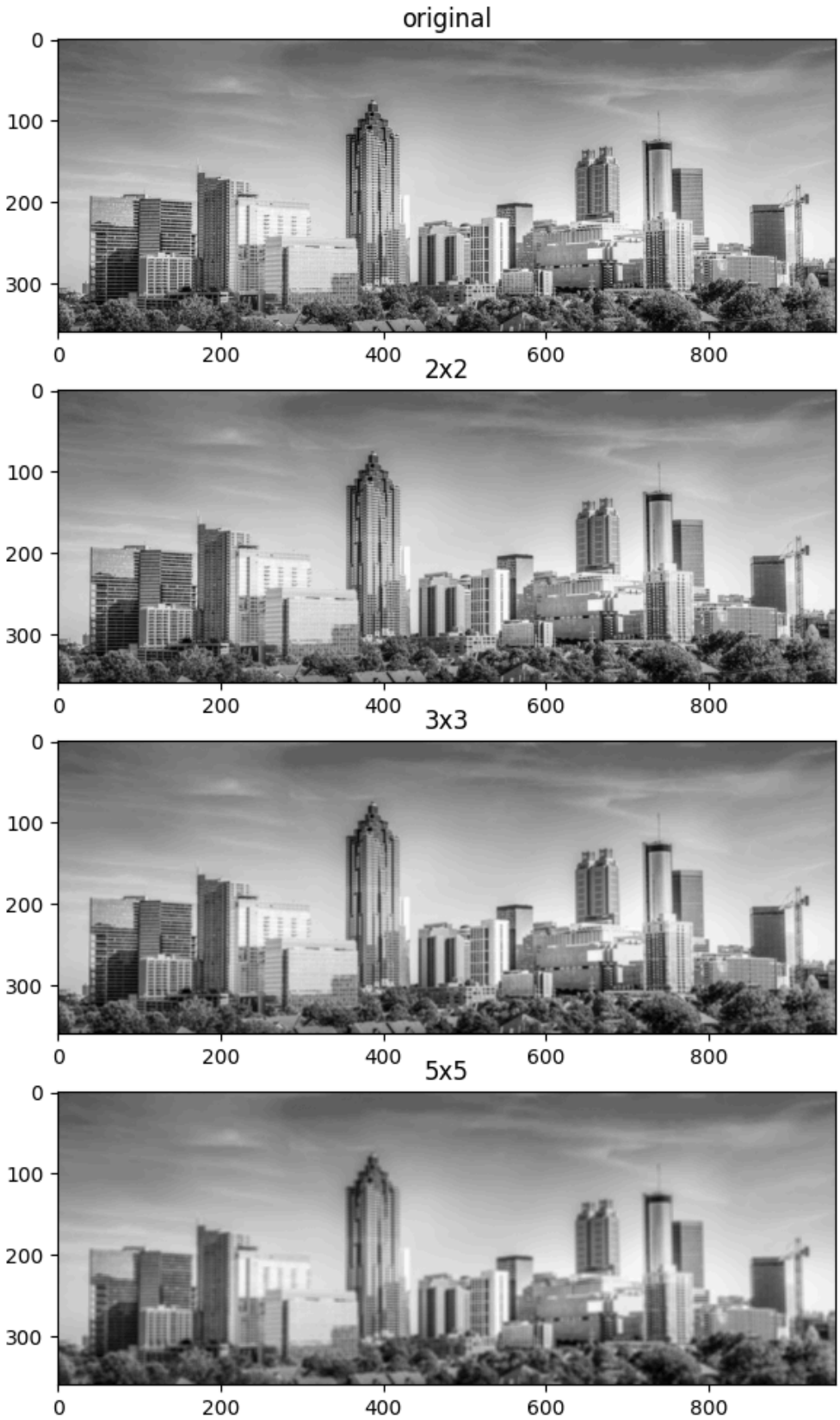
I included screenshots of the original image along with the horizontal, vertical, diagonal, and blur filter results to show how each kernel changed the image.

## Original Image

## Custom Kernel



**Custom Kernel for Blurring**




# 3. Part 2: CIFAR-10 Image Classification with CNN

## 3.1 Overview

In Part 2 of the assignment, I built a convolutional neural network using PyTorch to classify images from the CIFAR-10 dataset. CIFAR-10 contains 60,000 small color images split into 10 categories:

- airplane
- automobile
- bird
- cat
- deer
- dog
- frog
- horse
- ship
- truck

The dataset was divided into training, validation, and testing sets. The CNN was trained on the training data and then evaluated using the test set.

---

## 3.2 CNN Architecture

The model I used includes convolutional layers, max pooling layers, fully connected layers, ReLU activation, and dropout.

### Model structure

```
Input: 3x32x32
→ Conv2d(3,16,3,padding=1)
→ ReLU
→ MaxPool2d(2,2)
→ Conv2d(16,32,3,padding=1)
→ ReLU
→ MaxPool2d(2,2)
→ Flatten
→ Linear(32*8*8, 256)
→ ReLU
→ Dropout(0.25)
→ Linear(256, 10)
```

### PyTorch implementation summary

- `conv1 = nn.Conv2d(3, 16, 3, padding=1)`
- `conv2 = nn.Conv2d(16, 32, 3, padding=1)`
- `pool = nn.MaxPool2d(2, 2)`
- `fc1 = nn.Linear(32 * 8 * 8, 256)`
- `fc2 = nn.Linear(256, 10)`
- `dropout = nn.Dropout(0.25)`

The output layer has 10 neurons because CIFAR-10 has 10 image classes.

---

## 3.3 Loss Function and Optimizer

For the loss function, I used cross-entropy loss:

```
criterion = nn.CrossEntropyLoss()
```

For optimization, I used the Adam optimizer:

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

I chose Adam because it performed better than SGD for this model and helped the network train more efficiently.

---

## 3.4 Training Setup

The model was trained using the following settings:

- batch size: 20
- number of epochs: 20
- optimizer: Adam
- learning rate: 0.001
- dropout: 0.25
- normalization: `Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`

Training was performed on CPU.

---

## 3.5 Training Time

I measured the total training time using Python's `time` module.

- total training time: 294.63 seconds

I used the following code to measure the training time:

```
import time
start = time.time()
# training loop
end = time.time()
print("Training time:", end - start)
```

---

## 3.6 Number of Trainable Parameters

I calculated the total number of trainable parameters in the model using:

```
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(total_params)
```

- total trainable parameters: 532202

This gives an idea of the size and complexity of the model.

---

## 3.7 Accuracy Results

The final test accuracy achieved by the model was **72%** on the CIFAR-10 test set, with a test loss of **0.805536**.

The model performed best on classes such as **automobile** and **ship**, both with **85%** accuracy. It also performed well on **frog** with **82%** accuracy and **horse** with **79%** accuracy. Lower accuracy was seen for more difficult classes such as **cat** with **53%** accuracy and **bird** with **56%** accuracy, which means those categories were harder for the model to tell apart.

### Class-wise accuracy

- airplane: 75% (751/1000)
- automobile: 85% (857/1000)

- bird: 56% (562/1000)
- cat: 53% (533/1000)
- deer: 68% (681/1000)
- dog: 61% (615/1000)
- frog: 82% (822/1000)
- horse: 79% (792/1000)
- ship: 85% (851/1000)
- truck: 77% (772/1000)

```
Epoch: 1        Training Loss: 0.848426         Validation Loss: 0.889348
Validation loss decreased (inf --> 0.889348).  Saving model ...
Epoch: 2        Training Loss: 0.796480         Validation Loss: 0.878477
Validation loss decreased (0.889348 --> 0.878477).  Saving model ...
Epoch: 3        Training Loss: 0.758460         Validation Loss: 0.873031
Validation loss decreased (0.878477 --> 0.873031).  Saving model ...
Epoch: 4        Training Loss: 0.723932         Validation Loss: 0.882617
Epoch: 5        Training Loss: 0.687623         Validation Loss: 0.921836
Epoch: 6        Training Loss: 0.668601         Validation Loss: 0.860979
Validation loss decreased (0.873031 --> 0.860979).  Saving model ...
Epoch: 7        Training Loss: 0.639578         Validation Loss: 0.883637
Epoch: 8        Training Loss: 0.623490         Validation Loss: 0.875622
Epoch: 9        Training Loss: 0.602269         Validation Loss: 0.899445
Epoch: 10       Training Loss: 0.584521         Validation Loss: 0.868978
Epoch: 11       Training Loss: 0.567735         Validation Loss: 0.884219
Epoch: 12       Training Loss: 0.542286         Validation Loss: 0.871779
Epoch: 13       Training Loss: 0.536841         Validation Loss: 0.912504
```

```
Test Loss: 0.805536

Test Accuracy of airplane: 75% (751/1000)
Test Accuracy of automobile: 85% (857/1000)
Test Accuracy of  bird: 56% (562/1000)
Test Accuracy of   cat: 53% (533/1000)
Test Accuracy of  deer: 68% (681/1000)
Test Accuracy of   dog: 61% (615/1000)
Test Accuracy of  frog: 82% (822/1000)
Test Accuracy of horse: 79% (792/1000)
Test Accuracy of  ship: 85% (851/1000)
Test Accuracy of truck: 77% (772/1000)

Test Accuracy (Overall): 72% (7236/10000)
```

---

## 3.8 What Helped and What Did Not Help

Based on the changes I made in the notebook, several choices helped improve the model's performance. I used the Adam optimizer with a learning rate of 0.001 instead of SGD, and this gave better training behavior for my model. I also trained the network for 20 epochs, which gave

the model more time to learn compared to a shorter training run. Adding dropout with a value of 0.25 likely helped reduce overfitting, especially before the fully connected layers. The two convolutional layers with max pooling were enough to produce a reasonable result on CIFAR-10, and the model reached 72% test accuracy.

At the same time, there were still some limitations in my implementation. I kept the normalization simple by using `(0.5, 0.5, 0.5)` for both mean and standard deviation instead of using CIFAR-10-specific normalization values. I also did not include data augmentation techniques such as random cropping or horizontal flipping, which could likely improve generalization. The model itself was still fairly small, with only two convolutional layers, so it had less capacity than a deeper CNN. Finally, training on CPU made the process slower and limited how much experimentation I could do.

Overall, the main changes I made were using Adam, training for 20 epochs, and adding dropout, and these choices helped the model achieve a solid accuracy for this assignment.

---

# 4. Conclusion

In this assignment, I worked with both traditional image filtering and deep learning methods. In Part 1, I used convolution kernels to detect edges and blur images, which helped me understand how filtering changes image features. In Part 2, I built and trained a CNN to classify CIFAR-10 images and observed how choices such as optimizer, dropout, and number of epochs affected the final accuracy.

Overall, this assignment gave me hands-on experience with both low-level image processing and CNN-based image classification, and it helped me better understand how convolution is used in both approaches.

---

# 5. References


- OpenCV Documentation: https://opencv.org/
- PyTorch Documentation: https://pytorch.org/docs/stable/index.html
- CIFAR-10 Dataset: https://www.cs.toronto.edu/~kriz/cifar.html